

AsymptoticAnalysis

Validating core components before cancellation

A reproduced remainder-contract failure, exact all-depth counterexamples, and a focused repair for the shared source

Principal result. The exact-core constructor checks whether `core + perturbation` is target-free, then uses the two components separately. Cancelling occurrences of `Re[y]` bypass that check. Two successful public calls for the equation $x + x^{-1} = y$ claim errors of $O(y^{-2})$ and $O(y^{-4})$, although both actual errors are $\Theta(y^{-1})$. A component-wise guard rejects the offending split without forbidding a legitimate target-dependent `CoreInverse` expression.

Repository	VladimirReshetnikov/Asymptotic
Pinned revision	efa1aeec4845a9c35e140963a0333d0c9ec33b05
Revision time	10 September 2026, 19:09:04 UTC
New finding families	One, with two public counterexamples
Native execution	Wolfram 15.0.0, Linux x86 (64-bit)
Mathics execution	Not performed; portable regression probe supplied
Independent checks	30 Python tests, all passing

Scope: broad, incremental mechanism-level review with targeted execution. This is not a full-suite acceptance record or a claim that every source line and every archived report paragraph has been independently verified.

Abstract

This report audits selected analytic engines, coordinate transformations, coefficient and remainder calculations, Mathics adapters, and result contracts at a pinned revision of AsymptoticAnalysis. The exclusion set comprises the repository’s six review-wave indexes, maintained finding register and intakes, and targeted original reports. After removing already recorded mechanisms and ruling out several plausible but incorrect candidates, the report retains one new high-priority correctness finding.

The defect is in the admission of the separate core and perturbation arguments to `AsymptoticCoreInverse`. Validation of their evaluated sum loses target occurrences that cancel, whereas the subsequent marker expansion retains those occurrences in the two components. Consequently a fixed-data asymptotic majorant is applied to moving data. Public Wolfram 15.0.0 executions reproduce false remainder classes at both an infinite and a finite source endpoint. Elementary quadratic inverses prove the errors exactly. A Catalan coefficient formula extends the diagnosis to every fixed marker depth and explains why marker convergence does not rescue the claimed power order.

A two-line candidate patch was exercised in a temporary, commit-pinned standalone copy: five focused public requests exhibited the expected refusals or preserved controls. Separately, 22 exact mathematical tests and eight patch-utility fixture tests pass. A 17-case native forward-expansion screen checks a different part of the package and is reported separately. The archive contains the article, patch, exact rational oracles, numerical enclosures, observation records, novelty ledger, source inventory, and unexecuted complete Wolfram/portable regression files. No Mathics execution, Windows-kernel acceptance, or full repository-suite execution is asserted.

Contents

1	Executive assessment	2
2	Baseline, source coverage, and non-overlap	3
3	N01: a wrong remainder at an infinite source endpoint	4
4	The same defect at a finite source endpoint	6
5	Source-level cause and the failed theorem premise	7
6	Exact all-depth analysis	8
7	Candidate repair and its precise contract	12
8	Validation record and limits	13
9	Wolfram 15 and Mathics3 implications	15
10	Specific API and development follow-through	16
11	Reproduction and archive guide	18
12	Conclusion	20
A	The native forward-screen inputs	20
B	Ruled-out and excluded candidates	21
C	Acceptance case map	21

1 Executive assessment

1.1 The retained new finding

N01 — target dependence erased by sum validation. The constructor accepts a core decomposition that its own fixed-data remainder theorem does not admit. The finite approximation and the remainder are both part of the returned analytic object; checking only that the forward equation simplifies to a target-free expression is insufficient. In the large-source witness, the first inverse correction is wrong by $1/(4y)$ at marker depth two. In the finite-source witness, even the leading coefficient is wrong: $7/8$ instead of 1. Both objects nevertheless carry much smaller error classes. The observed source mechanism is at lines 126–127 of `src/Kernel/CorePerturbation.wl`; the downstream bound is formed in the same module [2].

The recommended immediate change is to test the tuple `{core, perturbation}`, not its evaluated sum. This is a local correctness repair. It neither broadens the supported asymptotic scale nor changes which parameters a user-supplied inverse is allowed to contain. The tuple-based `FreeQ` check alone closes the reproduced target leak; the accompanying tuple-based general input validation aligns the validation object with the data actually consumed.

1.2 Why one finding can justify a substantial incremental report

The pinned tree indexes 55 numbered reports across six waves, with 45 retained packages and ten retirement tombstones. Numerous earlier correctness, performance, API, and portability issues are already recorded. Those are not repackaged here as new discoveries [5, 6, 7, 8, 9, 10]. The useful deliverable is therefore a sharply delimited new failure, with a proof, reproducible evidence, a small repair, and acceptance obligations that are specific to that repair.

The source examination was broader than the retained finding. It included ordinary forward expansions and observables; Fourier and logarithmic coefficient paths; flat and exact-core sectors; Gamma/Barnes inverses; Dirichlet and parameterized special-function paths; coordinate reconstruction; interval-certificate logic; and selected Mathics proof/algebra/Taylor adapters. The machine-readable inventory records whether a file was read completely or only in relevant ranges. No conclusion of global correctness follows merely because no additional unreported defect was retained.

1.3 Decision summary

Decision	Basis and boundary
Repair now	N01 has two successful public reproductions, exact counterexamples, and a component-local admission fix.
Keep the valid API	A fixed real core parameter remains admissible; a supplied <code>CoreInverse</code> must still be allowed to depend on the target.
Do not infer Mathics acceptance	The source repair is shared, but no fresh Mathics package execution was available. Earlier realness checks may alter public reachability there.
Do not claim a performance defect	Known dense-allocation and recurrence issues were excluded. No new measured end-to-end slowdown is established here.
Do not widen the theorem silently	Moving core parameters need a different joint asymptotic argument, not just a successful symbolic marker calculation.

2 Baseline, source coverage, and non-overlap

2.1 The exact reviewed artifact

The repository revision is `efa1aeec4845a9c35e140963a0333d0c9ec33b05`, recorded at 19:09:04 UTC on 10 September 2026. The canonical implementation resides in `src/Kernel/`; the root `Asymptotic Analysis.wl` is the generated, self-contained entry. Native probes downloaded that entry at the full commit identifier rather than following a moving `main` URL [1, 3, 4].

A local repository checkout was not obtained in this environment. Source was read through the GitHub connector, and the native execution service downloaded the pinned standalone file independently. This distinction matters: reading source excerpts, loading the entire generated package, and exercising selected public calls are three different activities. The successful native load returned `Null` and produced the expected ordinary inverse smoke result. No repository file, branch, or remote review directory was changed.

2.2 Coverage by mechanism

Area	Examination and outcome
Exact-core inversion	Model construction, automatic inverse, marker recurrence, input validation, output fields, and fixed-data majorant examined. N01 retained.
Ordinary forward calculus	Selected standalone definitions plus 17 native coefficient comparisons. A varying-coefficient observable control transports its lost precision correctly.
Fourier coefficient engine	Source ranges through construction, coefficient, and residual APIs; finite-endpoint power convention checked against the guide. That suspected power bug was ruled out.
Flat and logarithmic sectors	Finite flat model/recurrence/majorant and selected logarithmic model/unit-recursion paths examined. Existing reviewed obligations were not reissued.
Gamma/Barnes and special functions	Coefficient recurrences, frontiers, parameterized Taylor/Frobenius ingress, selected real-domain rules, and Dirichlet formulas inspected at the scope in the inventory.
Coordinate transformations	Target-log construction and selected source-chart reconstruction and observable contracts inspected.
Exact certificates	Selected interval arithmetic and source-domain proof paths examined. This report does not claim a new certificate soundness finding.
Mathics compatibility	Assumption, algebra, core-function, special-function, and Taylor adapters inspected. Runtime success is not inferred from source inspection.
Prior review corpus	Six wave indexes, maintained status/intakes, and targeted nearest original reports used for mechanism-level exclusion. Not a word-for-word rereading of every retained article.

The 32-entry `evidence/source-inventory.csv` supplies pinned URLs and file-specific inspection boundaries. Some broad files were fetched in full but read in selected relevant sections; the inventory deliberately does not label that as a complete line-by-line audit.

2.3 Nearest earlier findings

The closest earlier result is report 46’s target-dependent **SourceShift** in **AsymptoticExponentialCoreInverse**, also carried by report 50. It applies the same essential mathematical principle—parameters called fixed must really be fixed—but concerns a different public API, option, validation line, normalized exponential amplitude, and repair [11, 10]. N01 uses no **SourceShift** and no nondefault option. Its target occurrence is hidden by cancellation across two positional arguments. The earlier option guard does not repair that test.

Report 43 F03 concerns a supplied inverse/formula retaining the eliminated source variable. N01 uses automatic core inversion and supplies no source-bearing **CoreInverse**. The C15 composition issue concerns a varying parameter captured while combining stored series; N01 occurs before a single exact-core result is constructed. Option-identity issues likewise do not explain the option-free counterexample [12, 6, 7, 10].

These distinctions are recorded in `evidence/novelty-ledger.csv`. They are not assertions that the fixed-data principle itself is novel mathematics. The new contribution is a previously unmatched admission mechanism and its public failure, together with the corresponding local remedy. An unindexed paragraph elsewhere in the historical archive could warrant additional attribution; the claimed exclusion scope is the reviewed indexes, registers, intakes, and targeted source reports, not every archived sentence.

3 N01: a wrong remainder at an infinite source endpoint

3.1 A minimal public request

Load the pinned package in a fresh native kernel, then evaluate:

```
s = AsymptoticAnalysis'AsymptoticCoreInverse[
  x - Re[y], Re[y] + 1/x,
  {x, Infinity}, {y, 2}
];
```

The two arguments add to

$$f(x) = x + \frac{1}{x}. \quad (1)$$

The target symbol has disappeared from this sum. In the successful Wolfram 15.0.0 execution, the result was a **GeneralizedSeries**, with the following actual projections:

```
Normal[s]
(* y - 1/(y + Re[y])
   - (Re[y] + 1/(y + Re[y]))/(y + Re[y])^2 *)

s["Remainder"]
(* PowerLogRemainder[1/(y + Re[y]), 2, 0] *)

s["RemainderScaleExpression"]
(* 1/(y + Re[y])^2 *)

s["Function"]
(* x + 1/x *)
```

The retained domain is

```

y + Re[y] > 0 &&
Element[1/(y + Re[y]), Reals] &&
0 < 1/(y + Re[y]) < 1/E

```

Every condition holds on the real ray $y > 3$. Thus the witness is not obtained by evaluating outside the result's declared domain. The domain is a branch and coordinate condition; it does not restore the missing fixed-parameter premise.

3.2 Exact comparison with the selected inverse

For $y > 2$, the inverse branch tending to $+\infty$ is

$$g_+(y) = \frac{y + \sqrt{y^2 - 4}}{2}, \quad g_-(y) = \frac{y - \sqrt{y^2 - 4}}{2} = \frac{1}{g_+(y)}. \quad (2)$$

Both identities follow directly from $x^2 - yx + 1 = 0$. The chosen large root is positive and exceeds one; the forward derivative $1 - x^{-2}$ is positive there, so there is no local branch ambiguity.

On $y > 3$, the returned approximation and scale simplify to

$$A_2(y) = y - \frac{3}{4y} - \frac{1}{8y^3}, \quad R_2^+(y) = \frac{1}{4y^2}. \quad (3)$$

The exact root expansion is

$$g_+(y) = y - \frac{1}{y} - \frac{1}{y^3} - \frac{2}{y^5} - \frac{5}{y^7} - \cdots. \quad (4)$$

Consequently,

$$A_2(y) - g_+(y) = \frac{1}{4y} + \frac{7}{8y^3} + \mathcal{O}(y^{-5}). \quad (5)$$

In particular,

$$\frac{|A_2(y) - g_+(y)|}{R_2^+(y)} \sim y \longrightarrow +\infty. \quad (6)$$

No fixed multiplicative constant makes the advertised remainder class true on a sufficiently far positive ray.

This is stronger than a numerical discrepancy at a moderate target. It is an asymptotic contradiction. Nor is it merely a conservative bound: a conservative bound would be larger than necessary, whereas this one is too small by an unbounded factor. The displayed finite expression is also not the correct ordinary inverse expansion through the advertised precision.

3.3 Why $\text{Re}[y]$ is significant

The first attempted request with $\mathbf{x-y}$ and $\mathbf{y+1/x}$ returned `Failure["UnprovedCoreData", ...]`. That is a negative candidate, not a public wrong-result witness. Replacing the cancelling target occurrences by $\text{Re}[y]$ exposes the actual missing condition: the realness test can admit the coefficient even though it is not fixed on the target approach. The function Re expresses real part; it does not assert independence from the symbol inside it [14].

The point is not to ask **Assumptions** to declare the target a parameter. The constructor already rejects source/target occurrences in its parameter assumptions. Rather, this request makes target dependence survive in data whose reality is otherwise acceptable. A test for real coefficients is not a substitute for the target-free test.

4 The same defect at a finite source endpoint

4.1 A second public manifestation, not a second finding

The complementary source chart gives an even sharper witness:

```
t = AsymptoticAnalysis'AsymptoticCoreInverse[
  1/x - Re[y], Re[y] + x,
  {x, 0}, {y, 2}
];
```

Again the forward function is $x + x^{-1}$. Here the source tends to 0^+ and the correct inverse is $g_-(y)$ from (2). The successful native result, simplified under $y > 3$, is

$$B_2(y) = \frac{1 + 5y^2 + 14y^4}{16y^5} = \frac{7}{8y} + \frac{5}{16y^3} + \frac{1}{16y^5}. \quad (7)$$

Its raw remainder scale is $(y + \text{Re } y)^{-4}$ and its domain is the same positive-core-coordinate condition as above. On the positive real ray,

$$R_2^-(y) = \frac{1}{16y^4}. \quad (8)$$

The true small root is

$$g_-(y) = \frac{1}{y} + \frac{1}{y^3} + \frac{2}{y^5} + \frac{5}{y^7} + \cdots,$$

so

$$g_-(y) - B_2(y) = \frac{1}{8y} + \frac{11}{16y^3} + \frac{31}{16y^5} + \mathcal{O}(y^{-7}). \quad (9)$$

Therefore

$$\frac{|g_-(y) - B_2(y)|}{R_2^-(y)} \sim 2y^3. \quad (10)$$

The finite approximation misses one eighth of the leading inverse coefficient, yet the object claims fourth-power decay of its error. Both this and the large-source example are recorded under N01 because they share the same admission and majorant mechanism.

4.2 Exact numerical enclosures

Table 2 uses an independent rational oracle, not additional package executions. For each rational y , integer square-root arithmetic constructs outward rational endpoints for $\sqrt{y^2 - 4}$ on a grid with 100 decimal places. Equation (2) then encloses the selected root and the true approximation error. The complete fraction endpoints are retained in `evidence/exact-enclosures.json`; the displayed decimals are rounded summaries, not themselves directed-rounding certificates.

For the large root, the positive Catalan tail implies a ratio strictly greater than y at the tabulated points. For the small root, the leading mismatch and positive subsequent coefficients give the cubic divergence. The proof is the asymptotic calculation, while the rational intervals provide independently checkable finite-target evidence.

Branch	y	True absolute error	Advertised scale	Error / scale
Large	5	$5.77122 \cdot 10^{-2}$	10^{-2}	5.77122
Large	20	$1.26100 \cdot 10^{-2}$	$6.25 \cdot 10^{-4}$	20.1760
Large	100	$2.50088 \cdot 10^{-3}$	$2.5 \cdot 10^{-5}$	100.035
Large	1000	$2.50001 \cdot 10^{-4}$	$2.5 \cdot 10^{-7}$	1000.0035
Small	5	$3.11922 \cdot 10^{-2}$	10^{-4}	311.922
Small	20	$6.33655 \cdot 10^{-3}$	$3.90625 \cdot 10^{-7}$	16221.6
Small	100	$1.25069 \cdot 10^{-3}$	$6.25 \cdot 10^{-10}$	$2.00110 \cdot 10^6$
Small	1000	$1.25001 \cdot 10^{-4}$	$6.25 \cdot 10^{-14}$	$2.000011 \cdot 10^9$

Table 2: Depth-two moving-core witnesses with $c = y$. Independent exact root and error enclosures; decimals shown to a convenient precision.

5 Source-level cause and the failed theorem premise

5.1 Validation observes different data from the algorithm

The relevant source fragment is:

```
validateInput[core + perturbation, limit];
If[x == y || ! FreeQ[core + perturbation, y],
  fail["InvalidVariables",
    "Use distinct source and target symbols, with no target symbol in the forward data."]];
...
model = corePerturbationModel[
  core, perturbation, x, coord, ell, ass];
```

The first two checks see the evaluated sum. The model sees the original separate argument values. With

$$F_0 = x - c, \quad H = c + x^{-1}, \quad c = \operatorname{Re} y,$$

the sum is target-free, but neither of the two consumed components is. The corresponding finite-source split is

$$F_0 = x^{-1} - c, \quad H = c + x.$$

No option grammar, native-backend fallback, series composition, supplied core inverse, or numerical root finder is required to reach the defect.

`FreeQ` tests an expression for matching subexpressions [13]. It cannot recover a target occurrence already removed from its argument by evaluation. The logical implication being assumed is invalid:

$$\operatorname{FreeQ}(F_0 + H, y) \not\Rightarrow \operatorname{FreeQ}(F_0, y) \wedge \operatorname{FreeQ}(H, y). \quad (11)$$

The tuple form preserves the distinction between the two argument values and therefore tests the property actually needed by the algorithm.

5.2 What the model computes

Let u be the positive source coordinate. In the large-source case $u = 1/x$; in the finite-source case $u = x$. In both cases, for a nonzero frozen c ,

$$F_0(u) = u^{-1} - c, \quad H(u) = c + u.$$

The leading source power of the core is $p = -1$. The perturbation has powers 0 and 1, so its minimum gap above the core is $\delta = 1$. The core offset is $-c$, and its local inverse coordinate is

$$u_0 = \frac{1}{y + c}. \quad (12)$$

For the default observable, the local observable power is $r_{\text{loc}} = -1$ at the infinite source and $r_{\text{loc}} = 1$ at the finite source.

The implementation forms the truncation pair

$$(r_{\text{loc}} + (N + 1)\delta, (N + 1)d), \quad (13)$$

where d is a logarithmic-degree bound. Here $d = 0$, so the stored scale is u_0^N in the large-source chart and u_0^{N+2} in the small-source chart. The source explicitly describes its majorant as a fixed-data existence result based on fixed core and finite higher-power perturbation parameters [2]. The successful result does not provide uniform estimates for $c = c(y)$.

5.3 Why a correct marker calculation is not enough

The marker algorithm differentiates with respect to the source coordinate, holding coefficient parameters fixed. Its finite coefficients are valid for that frozen-parameter formal calculation. After the target-dependent split is admitted, however, those coefficients are specialized along a moving path in parameter space. In these witnesses the supposedly constant term c is of order y , not of fixed magnitude.

Equivalently, the normalized constant perturbation is comparable to $c/(y + c)$. When $c = y$, this is $1/2$, not a quantity tending to zero. Each marker order can reduce a coefficient geometrically without improving its power of y . An implicit-function theorem at fixed parameters does not imply the power remainder claimed after that substitution. Section 6 makes this precise without relying on numerical experimentation.

5.4 Severity and containment

N01 is a high-priority mathematical correctness issue because a successful analytic result carries a false error class. Its trigger is an unsupported moving split rather than an ordinary fixed decomposition. That makes the repair small, but it does not make the current success sound: the constructor must enforce the premise it advertises.

The report does not assert that a particular downstream interval certificate has accepted the bad result or that every runtime reaches the same branch. The demonstrated failure is already the returned remainder itself. The native witness and exact inverse proof suffice to establish that narrower conclusion.

6 Exact all-depth analysis

6.1 Freeze the parameter before introducing the moving path

Initially let $y > 2$ and $c > 0$ be separate fixed real numbers, and write $A = y + c$. Introduce a marker ε multiplying the perturbation. The large-source split gives

$$X - c + \varepsilon(c + X^{-1}) = y,$$

or

$$X^2 - (A - \varepsilon c)X + \varepsilon = 0, \quad X(0) = A. \quad (14)$$

The finite-source split gives

$$U^{-1} - c + \varepsilon(c + U) = y,$$

or

$$\varepsilon U^2 - (A - \varepsilon c)U + 1 = 0, \quad U(0) = A^{-1}. \quad (15)$$

These equations independently encode the frozen-parameter marker problem that the source recurrence solves. They need no implementation of `GeneralizedSeries` to analyze.

Their analytic branches at $\varepsilon = 0$ are

$$X(\varepsilon) = \frac{A - \varepsilon c + \sqrt{(A - \varepsilon c)^2 - 4\varepsilon}}{2}, \quad (16)$$

$$U(\varepsilon) = \frac{2}{A - \varepsilon c + \sqrt{(A - \varepsilon c)^2 - 4\varepsilon}}. \quad (17)$$

In particular,

$$X(\varepsilon)U(\varepsilon) = 1. \quad (18)$$

At $\varepsilon = 1$, the parameter c cancels from the equation, leaving precisely g_+ and g_- . The finite Taylor polynomials, however, still depend on the chosen decomposition.

6.2 General coefficient formulas

Define the Catalan numbers by

$$C_m = \frac{1}{m+1} \binom{2m}{m}, \quad m \geq 0.$$

The generating identity

$$\frac{1 - \sqrt{1 - 4z}}{2z} = \sum_{m \geq 0} C_m z^m$$

follows by expanding the square root or solving its quadratic formal equation. The following formulas are exact identities for Taylor coefficients; they are not numerical fits.

Proposition 6.1 (Frozen-parameter marker coefficients). *Write $X(\varepsilon) = \sum_{k \geq 0} L_k \varepsilon^k$ and $U(\varepsilon) = \sum_{k \geq 0} S_k \varepsilon^k$, with the branches above. Then $L_0 = A$, and for $k \geq 1$,*

$$L_k = -c \mathbf{1}_{k=1} - \sum_{m=1}^k C_{m-1} \binom{k+m-2}{k-m} \frac{c^{k-m}}{A^{k+m-1}}. \quad (19)$$

For every $k \geq 0$,

$$S_k = \sum_{m=0}^k C_m \binom{k+m}{k-m} \frac{c^{k-m}}{A^{k+m+1}}. \quad (20)$$

Proof. Put $B = A - \varepsilon c$. Applying the Catalan identity to (16)–(17) gives the formal series

$$X(\varepsilon) = B - \sum_{m \geq 1} \frac{C_{m-1} \varepsilon^m}{B^{2m-1}}, \quad U(\varepsilon) = \sum_{m \geq 0} \frac{C_m \varepsilon^m}{B^{2m+1}}.$$

For a positive integer j ,

$$(A - \varepsilon c)^{-j} = \sum_{\ell \geq 0} \binom{j + \ell - 1}{\ell} \frac{c^\ell}{A^{j+\ell}} \varepsilon^\ell.$$

Taking the coefficient of ε^k in each expression gives (19) and (20). The extra $-c$ occurs only in L_1 because $B = A - \varepsilon c$ is linear in the marker. $\square$

The initial coefficients illustrate the calculation:

$$\begin{aligned} L_0 &= A, & L_1 &= -c - A^{-1}, & L_2 &= -cA^{-2} - A^{-3}, \\ S_0 &= A^{-1}, & S_1 &= cA^{-2} + A^{-3}, & S_2 &= c^2A^{-3} + 3cA^{-4} + 2A^{-5}. \end{aligned}$$

Their sums through degree two, with $c = y$, give exactly the two native finite expressions (3) and (7). The independent Python implementation verifies the quadratic equations and reciprocal coefficient convolution at several exact parameter choices. The proof above, not those finitely many tests, establishes the general formulas.

6.3 Specialize to a moving core

Only now set

$$c = \lambda y, \quad \lambda > 0, \quad \theta = \frac{\lambda}{1 + \lambda} \in (0, 1). \quad (21)$$

Define the depth- N approximations at marker value one by

$$A_N(y) = \sum_{k=0}^N L_k|_{c=\lambda y}, \quad B_N(y) = \sum_{k=0}^N S_k|_{c=\lambda y}.$$

The following is an all-depth deduction from the inspected recurrence and independent quadratic equations. The public native observations themselves were the $\lambda = 1$, $N = 2$ cases.

Theorem 6.2 (Wrong orders at every fixed affected depth). *Fix $\lambda > 0$. For each fixed $N \geq 1$,*

$$A_N(y) - g_+(y) = \frac{\theta^N}{y} + \mathcal{O}(y^{-3}). \quad (22)$$

For each fixed $N \geq 0$,

$$g_-(y) - B_N(y) = \frac{\theta^{N+1}}{y} + \mathcal{O}(y^{-3}). \quad (23)$$

The scales inferred by the current fixed-gap construction for nonzero c are A^{-N} and $A^{-(N+2)}$, respectively. Their error ratios satisfy

$$\frac{|A_N - g_+|}{A^{-N}} \sim \lambda^N y^{N-1}, \quad N \geq 1, \quad (24)$$

$$\frac{|g_- - B_N|}{A^{-(N+2)}} \sim \lambda^{N+1} (1 + \lambda) y^{N+1}, \quad N \geq 0. \quad (25)$$

Thus the large-source scale is false for every $N \geq 2$, and the finite-source scale is false for every $N \geq 0$.

Proof. For fixed k , the $m = 1$ term in (19) is

$$-\frac{c^{k-1}}{A^k} = -\frac{1}{(1+\lambda)y}\theta^{k-1}.$$

Every term with $m \geq 2$ is $O(y^{-3})$, since its power of y is $1 - 2m$. For $N \geq 1$, the $A - c$ contribution is y . Summing the geometric progression yields

$$A_N(y) = y - \frac{1 - \theta^N}{y} + O(y^{-3}).$$

Subtract (4) to obtain (22).

In (20), the $m = 0$ term is $c^k/A^{k+1} = \theta^k/((1+\lambda)y)$. All $m \geq 1$ terms are $O(y^{-3})$. Therefore

$$B_N(y) = \frac{1 - \theta^{N+1}}{y} + O(y^{-3}),$$

which proves (23). Finally $A = (1+\lambda)y$ and the source gap is one, so division by the two advertised scales gives (24)–(25). $\square$

Remark 6.3 (Depth and zero-parameter exceptions). For the large-source branch at $N = 1$, the ratio tends to λ , so this particular scale is not refuted at that depth. At $N = 0$, the large approximation is $(1+\lambda)y$ and its error is $\lambda y + O(y^{-1})$, while the inferred scale is one; that depth also fails. These cases should not be blurred into an incorrect claim about all depths having the same behavior.

When $c = 0$, the constant perturbation row disappears. The minimum source gap is then two, not one. The valid unshifted construction consequently uses scales $y^{-(2N+1)}$ for the large branch and $y^{-(2N+3)}$ for the small branch. For example, the patched native large-source depth-two control returns $y - y^{-1} - y^{-3}$ with scale y^{-5} . Substituting $\lambda = 0$ into a nonzero-row analysis would miss this structural change.

6.4 Convergence in the marker does not repair the power claim

There is a particularly instructive distinction here. For fixed $c > 0$ and $A > 0$, the nearest positive discriminant zero of the exact marker functions is

$$\varepsilon_- = \frac{(\sqrt{Ac+1}-1)^2}{c^2}. \quad (26)$$

This follows by solving $(A - \varepsilon c)^2 - 4\varepsilon = 0$; both branch points are positive, and the smaller one is (26). The branch at zero is analytic up to that nearest square-root singularity. Under $c = \lambda y$,

$$\varepsilon_- \longrightarrow \frac{1+\lambda}{\lambda} > 1 \quad (y \rightarrow \infty).$$

For the actual $c = y$ witnesses, $\varepsilon_- > 1$ whenever $y > 2$. Hence their Taylor series in the marker genuinely converge at $\varepsilon = 1$ on the ray used in the counterexample.

There is no contradiction with Theorem 6.2. Sending the marker depth to infinity at a fixed target and sending the target to infinity at a fixed marker depth answer different questions. A residual leading coefficient of size θ^N can shrink geometrically with N but remain attached to y^{-1} . This example therefore isolates an invalid asymptotic grading, not necessarily a divergent marker algorithm.

One can also see the warning in the first omitted large-source marker coefficient. Formula (19) gives, at $c = y$,

$$L_3 = -\frac{1}{8y} - \frac{3}{16y^3} - \frac{1}{16y^5}. \quad (27)$$

That term alone is not $O(y^{-2})$. Equation (27) is a source-derived independent formula; a later native attempt to project the stored omitted-term field failed at the service level, so no native observation of that field is claimed.

7 Candidate repair and its precise contract

7.1 The two-line change

The supplied canonical-source patch is:

```
- validateInput[core + perturbation, limit];
- If[x === y || ! FreeQ[core + perturbation, y],
+ validateInput[{core, perturbation}, limit];
+ If[x === y || ! FreeQ[{core, perturbation}, y],
  fail["InvalidVariables", ...]];
```

The second changed line is the essential soundness repair for N01. With an ordinary list preserving both values, `FreeQ` can see a target occurrence in either component even if the sum would cancel it. This uses the existing structural policy; it does not attempt a new lexical free-variable analyzer for held functions or binders.

The first changed line checks exactness, nonfinite content, and the generic input budget on the same component tuple that the algorithm will consume. It is a consistency hardening alongside the target guard, not a second independently demonstrated defect. In particular, the native baseline already rejects the attempted inexact cancellation control. That observation must not be recast as proof of a new inexact-admission hole.

Tuple validation can charge a slightly different leaf count from sum validation, especially where large subexpressions cancel. This is appropriate for an algorithm that retains the separate components, but it is a budget-policy change worth acknowledging. The minimal target-only variant leaves the first line unchanged and still closes the reproduced counterexamples. The tested candidate and supplied diff use both replacements.

7.2 Preserve intentional dependencies

The target is not forbidden from every argument. A supplied `"CoreInverse" -> phi[y]` is expected to be a function of the target; the existing source-independence and branch-identity checks for that expression remain separate. A repair that adds `requested` to the target-free tuple would reject legitimate use.

The patched native control

```
AsymptoticAnalysis[AsymptoticCoreInverse[
  x - 1, 1 + 1/x, {x, Infinity}, {y, 2},
  "CoreInverse" -> y + 1
]
```

returns a successful analytic result with finite expression

$$y - \frac{1}{y+1} - \frac{1}{(y+1)^2} - \frac{1}{(y+1)^3}$$

and remainder scale $(y + 1)^{-2}$. The native difference from this expected expression simplified exactly to zero. Here the core offset is truly fixed. This is not the moving $c = y$ family.

7.3 Patch-utility behavior

`code/patch_core_validation.py` emits a unified diff by default. It checks that each old literal anchor occurs exactly once, and refuses a missing or duplicate anchor or an already-patched file. Its optional `-output` argument creates a separate, previously nonexistent file; it never overwrites the input or an existing destination. Eight fixture tests exercise these properties. They are tests of the utility, not tests of the package.

The utility does not prove the input file's Git provenance, install the package, edit the user's checkout, or rebuild the standalone distribution. Use the pinned revision and inspect the emitted diff. After integrating the canonical module change, regenerate the root standalone with the repository's maintained builder [3]. Maintaining a hand-edited generated fork would make later source reconciliation unnecessarily difficult.

7.4 Native patch observations

The execution service imported the pinned standalone as text. Both replacement anchors occurred once. Two literal replacements were applied, the result was written to a temporary file, and a fresh native kernel loaded that file. No canonical repository source was changed. Across two successful queries, the following five public requests were observed:

Case	Request class	Observed patched outcome
CV01	Large-source cancelling <code>Re[y]</code>	<code>Failure["InvalidVariables",...]</code>
CV03	Finite-source cancelling <code>Re[y]</code>	<code>Failure["InvalidVariables",...]</code>
CV08	Unshifted large source: core x , perturbation $1/x$, depth two	$y - y^{-1} - y^{-3}$, scale y^{-5}
CV10	Fixed offset with explicit <code>CoreInverse -> y+1</code>	Expected expression preserved; scale $(y + 1)^{-2}$
CV12	Inexact-component negative control, depth one	<code>Failure["InexactInput",...]</code>

These are five focused observations, not an executed twelve-test MUnit file, a full-suite result, or Mathics acceptance. Their manual transcription is in `evidence/native-observations.json`.

8 Validation record and limits

8.1 Native environment and failures

The successful native service reports

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

The service is stateless across queries; each package query loaded its own commit-pinned entry. Several connection and HTTP-502 attempts returned no usable result. Those failures are excluded from every

observation count. They neither prove that a package call failed nor establish that its expected output occurred.

The durable native record is a manual transcription of successful connector responses, not a machine-signed or automatically replayed receipt. It preserves that provenance explicitly. No successful Mathics session or local Wolfram executable was available, and the full repository test suite was not run.

8.2 A separate native forward screen

Seventeen ordinary forward inputs were expanded with

```
s = AsymptoticAnalysis'AsymptoticExpansion[
  f, {x, 0, 4}, "Backend" -> "Package"
];
FullSimplify[Normal[Series[f - Normal[s], {x, 0, 3}]]]
```

Every input returned a `GeneralizedSeries`, and all seventeen displayed differences were zero. The inputs cover pole-amplified Taylor data, cancellation before a singular quotient, bounded composite arguments, exact irrational powers, and varying powers such as $(1+x)^{1/x}$. Appendix A lists them with their returned remainder powers.

This is useful positive evidence for the exercised coefficient paths. It does not certify every higher remainder order the objects reported: the comparison was through powers below four. It is also not an independent implementation of the native algebra system. It compares the package's output against the native series of its difference with the source.

One additional observable control was successful:

```
s = AsymptoticAnalysis'AsymptoticExpansion[
  Sin[x], {x, 0, 4}, "Backend" -> "Package"];
AsymptoticAnalysis'SeriesObservable[s, z/x^4, z]
(* finite expression: 1/x^3 - 1/(6 x)
   remainder: PowerLogRemainder[x, 1, 0] *)
```

Thus the varying multiplier transports the sine remainder to the expected first power in this case. A suspicion that it was always treated as a constant was not retained.

8.3 Independent exact tests

The Python companion uses only the standard library. It does not call Wolfram, Mathics, SymPy, or the package. Its run completed with **30 test methods passed, zero failures, zero errors, zero skips** on Python 3.13.5. These split into 22 exact mathematical checks and eight patch-utility fixture checks.

The mathematical checks include the Catalan coefficient formulas, the two implicit quadratic equations through order ten at selected exact parameters, and the reciprocal convolution (18) through order eight. They compare the depth-two formulas with the native observations, test the unshifted Catalan specialization and its changed source gap, verify outward root brackets, and examine the predicted leading-error constants at several fixed depths and positive rational λ . The all-depth tests at a large exact target are finite checks of a proved formula, not proofs of a limit by sampling.

To enclose a rational square root $\sqrt{a/b}$, the oracle sets $D = 10^d$ and

$$k = \text{isqrt}\left(\left\lfloor aD^2/b \right\rfloor\right).$$

Then $k/D \leq \sqrt{a/b} < (k+1)/D$, except that an exact equality is retained when $(k/D)^2 = a/b$. These inequalities use integers and rational arithmetic only. The small-root formula $2/(y + \sqrt{y^2 - 4})$ avoids subtracting close endpoints. The resulting rational error enclosures are independent of floating-point roundoff and of the package's own remainder claims.

The oracle is deliberately narrow. Its bounded depth and rational-input checks are engineering safeguards, not a general resource sandbox or a new asymptotic package. It is an executable mathematical counterexample and regression reference for this audit.

8.4 Supplied acceptance files versus executed observations

The archive includes a twelve-case desired post-patch MUnit file and a four-case no-MUnit portable probe. Neither complete file was executed during this audit. Some corresponding requests were executed inline, as specified in the patch table, but that does not make the whole file an acceptance receipt.

The twelve cases add controls for `Abs[y]`, plain `y`, a target in only one component, identical source/target variables, a regular finite-endpoint example, and a fixed real symbolic parameter. The four portable cases focus on the two bad splits, the unshifted control, and the legitimate supplied inverse. Each file requires the package to have been loaded first. Public package names are fully qualified, and the portable file avoids dependence on MUnit.

Evidence category	What was actually established
Public baseline counterexamples	Two native successful objects; exact analysis refutes their returned remainder scales.
Patched native requests	Five requested outcomes observed in temporary patched standalone copies.
Native forward screen	Seventeen coefficient differences below power four vanish; no exhaustive remainder certification.
Independent Python tests	Thirty methods pass; none loads the package.
Supplied MUnit / portable files	Twelve / four desired cases supplied; complete files not executed.
Mathics / full repository suite	Not executed.

No pooled success total is meaningful across these categories.

9 Wolfram 15 and Mathics3 implications

9.1 What transfers at the source level

The defective guard is in shared canonical source, not solely in a Wolfram backend adapter. Both public argument values must be target-free for the fixed-data exact-core construction, regardless of which evaluator eventually executes it. The proposed list-based check uses the same existing `FreeQ` policy and ordinary list structure rather than a runtime-specific symbolic solver. This makes it a natural shared-source repair.

It does not follow that the unpatched Mathics package must return the same bad objects. Earlier realness, sign, automatic-core, or simplification behavior may refuse a candidate before the faulty majorant is exposed. Such a refusal would be safe for that request, not proof that the shared missing guard is correct. The correct Mathics statement from this audit is therefore *shared source obligation, fresh public execution unverified*.

9.2 A focused two-runtime acceptance target

For this repair, the decisive behavior is simple: both target-cancelling splits must be rejected by the dependency check, and fixed-data controls must remain successful where that runtime supports the underlying operations. The supplied portable probe provides four concrete cases rather than treating package load as a compatibility claim. The native MUnit file adds a wider twelve-case contract matrix.

The modular and generated standalone entries should both contain the repaired guard after rebuilding. Run the same focused cases in fresh Wolfram 15 and Mathics sessions, record the actual runtime/version and entry, and retain a structured distinction between the intended dependency refusal and an earlier unrelated capability refusal. This is an acceptance obligation for N01, not a reissued recommendation for a wholesale new CI or compatibility architecture.

The user guide already describes complete Mathics support and complete native expansion coverage as unfinished goals [4]. This report does not add those familiar gaps as new findings, and it does not infer Windows 15.0.1 behavior from the Linux 15.0.0 observations.

9.3 Performance and resource interpretation

No new end-to-end timing or memory defect is claimed. The target-free tuple check traverses the actual retained inputs rather than relying on their possibly simplified sum. Its intended benefit is sound admission, not a measured speedup. It can also avoid spending downstream symbolic work on an invalid split, but no benchmark quantifies that effect here.

The independent reference implementation is fast for these small finite quadratics; that timing says nothing about package performance. Existing reports already cover dense sparse-polynomial allocations, repeated symbolic work, recurrence termination, and runner resource concerns. Their recommendations are excluded rather than relabeled as opportunities found in this audit [6, 8, 10].

10 Specific API and development follow-through

10.1 Make the rejected component explicit

The present failure message says that forward data must not contain the target, but users can see a target-free stored sum and reasonably believe that condition holds. After the repair, a useful focused diagnostic improvement is to identify which positional component contains the forbidden target. For example, retain `Failure["InvalidVariables",...]` while adding `"TargetVariable" -> y` and `"DependentComponents" -> {"Core", "Perturbation"}`. A compact component-name list is preferable to duplicating a potentially large expression in the failure data.

This diagnostic is a proposal, not part of the two-line patch or its five native observations. It should be derived from the same component values used by the guard, without an extra simplifying sum or

repeated evaluation of user expressions. The current tag can remain stable for callers that dispatch on it.

10.2 Document three different dependency roles

The exact-core section of the guide should state the newly enforced boundary in concrete terms: each of the core and perturbation arguments must be independent of the requested target; cancellation between them does not satisfy that condition. A supplied inverse is instead expected to depend on the target but not on the eliminated source. Parameter assumptions refer to fixed parameters under the existing policy.

A useful documentation pair is the refused moving split $x - \text{Re}[y]$, $\text{Re}[y] + 1/x$ beside the accepted fixed split $x - 1$, $1 + 1/x$ with "CoreInverse" $\rightarrow y + 1$. That example explains the actual distinction much better than a blanket statement that all auxiliary data must be target-free. This is a proposed clarification specific to N01, not a claim that every related API should receive identical restrictions.

10.3 Use sum-preserving input transformations as tests

The mechanism suggests a small, reproducible regression family. Starting with a valid decomposition (F_0, H) , replace it by

$$(F_0 - C, H + C). \quad (28)$$

The forward sum is unchanged, but the marker decomposition is changed. With a fixed admissible constant C , the constructor may legitimately return a different finite marker approximation with its corresponding bound. Equality of finite approximations at equal marker depth must *not* be imposed. With $C = \text{Re } y$ or another retained target-dependent expression, the current API should reject the transformed request before theorem-based construction.

This tests the information-loss boundary directly. It distinguishes a true sum-invariant property—the underlying forward equation—from a split-dependent property—finite marker coefficients and their proof premises. The supplied native tests instantiate both sides of that distinction.

10.4 Retain the reciprocal marker oracle

For the two quadratic charts, the coefficient identity

$$\sum_{j=0}^k L_j S_{k-j} = \begin{cases} 1, & k = 0, \\ 0, & k > 0 \end{cases}$$

is an exact cross-chart check. Together with each defining quadratic equation, it can detect a recurrence or observable reconstruction error independently of public formatting and remainder metadata. It is already implemented in the companion exact tests.

This invariant is deliberately restricted to the stated paired family. It is not proposed as a universal inverse-residual checker or an excuse to accept moving coefficients. Passing the algebraic identity proves a property of the marker coefficients, while target-independence remains a separate admission obligation.

10.5 Moving cores are outside this repair

The exact formulas show that a moving core can be algebraically meaningful and can even have a convergent marker series. Supporting it with useful asymptotic error statements would nevertheless require an explicit joint grading or a uniform parameter-dependent bound. A realness proof, successful simplification, or small numerical marker residual is not that theorem.

For the current API, reject the unsupported moving split. Replacing the result by an automatically chosen different core would change the user's requested marker decomposition and should not happen silently. A future explicitly separate mode could choose a different contract, but this report makes no claim that such a mode has been designed, implemented, or validated. The immediate repair should remain small and auditable.

11 Reproduction and archive guide

11.1 Rebuild the article

The article is self-contained LaTeX using standard text, mathematics, table, and listing packages. No external figure or font file is required:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error review.tex
pdflatex -interaction=nonstopmode -halt-on-error review.tex
# Repeat once more if cross-reference warnings remain.
```

The delivered PDF was compiled from this source. The build summary and layout checks are recorded separately in the evidence directory.

11.2 Run the independent mathematics and patch tests

From the extracted archive root:

```
python code/run_checks.py
```

Python 3.10 or later is required by the companion type syntax; the recorded run used 3.13.5. There are no third-party Python dependencies. The command writes its transcript, summary, exact enclosures, and decimal sample table into `evidence/`. It does not execute or download the Wolfram package. Those new local outputs replace the companion run records, not repository files.

11.3 Prepare the canonical patch

With a checkout of the pinned source:

```
python code/patch_core_validation.py \
/path/to/Asymptotic/src/Kernel/CorePerturbation.wl
```

The default output is a diff for inspection. To create a distinct candidate file without overwriting the source:

```
python code/patch_core_validation.py \
/path/to/Asymptotic/src/Kernel/CorePerturbation.wl \
--output /new/path/CorePerturbation.patched.wl
```

The output path must not already exist. Integrate the reviewed changes into the canonical module using the usual repository workflow, then run the maintained standalone builder from the repository root:

```
python validation/build_standalone.py
```

No such integration into the user's repository was performed in this audit.

11.4 Run package observations and acceptance

In a fresh Wolfram kernel, first load a selected local package entry, then load the observation file or run the post-patch MUnit file as separate inputs:

```
Get["/path/to/AsymptoticAnalysis.wl"];
Get["/path/to/audit/code/ProbeComponents.wl"];

(* For the patched package: *)
TestReport["/path/to/audit/code/ReviewComponents.wlt"]
```

For a Mathics session, load the package first, using its maintained runtime instructions and evaluator budget, then evaluate:

```
Get["/path/to/audit/code/PortableComponents.wl"];
```

Do not treat this instruction as a recorded successful Mathics run. The probe prints a runtime-tagged result with passed and failed case counts. It contains no automatic network download, repository edit, or global replacement of package definitions.

11.5 Files and provenance

File or group	Purpose
article/review.tex, review.pdf	Complete article source and compiled report.
code/component-validation.diff	Two-line canonical-source candidate.
code/patch_core_validation.py	Checked, non-overwriting diff/copy utility.
code/reference.py	Independent exact quadratic/marker/root oracle.
code/test_independent.py, run_checks.py	Thirty independent mathematics and patch-fixture test methods and evidence generator.
code/ReviewComponents.wlt	Twelve desired post-patch native acceptance cases; complete file unexecuted in the audit.
code/PortableComponents.wl	Four no-MUnit regression cases; Mathics execution unverified.
code/ProbeComponents.wl	Public observation probe for baseline or patched source.
code/ForwardScreen.wl	Reproducible case list for the separately executed native coefficient screen.

File or group	Purpose
evidence/native-observations.json	Manual transcription of successful native responses and explicit execution limits.
evidence/exact-enclosures.json, numerical-samples.csv	Independent rational root/error endpoints and rounded decimal summaries.
evidence/independent-tests.*	Actual Python test transcript and summary.
evidence/novelty-ledger.csv, source-inventory.csv, scope.json	Non-overlap rationale, inspected-source coverage, and provenance boundaries.

The ZIP contains no checksum files, full repository mirror, bundled third-party libraries, font files, or TeX auxiliary build products. Commit identifiers identify the source baseline; they are not standalone checksum files.

12 Conclusion

The new defect is an information-loss error at an admission boundary: validation observes an evaluated sum, whereas the algorithm and its theorem depend on the separate summands. The two public counterexamples are elementary enough to exclude uncertainty about special-function conventions, numerical solvers, branch selection, or hidden constants. Their exact inverses make the false remainder statements unavoidable.

The corrective action is equally specific. Preserve the components while checking their target independence, preserve the distinct dependency role of a supplied inverse, regenerate the standalone from canonical source, and execute the focused component tests on both supported runtimes. The tested candidate already rejects both native witnesses and preserves selected valid controls; full Mathics and broader acceptance remain unexecuted obligations, not assumed successes.

Finally, the all-depth calculation explains why a plausible formal computation can still produce an invalid analytic object. Convergent marker coefficients, a target-free final equation, and a real local coordinate do not by themselves supply a fixed-data asymptotic remainder. Enforcing that missing premise is the substantive contribution of this incremental audit.

A The native forward-screen inputs

All cases used an exclusive package cutoff of four at $x \rightarrow 0^+$ and an explicit "Backend"-"Package". All seventeen native comparisons of the source minus the finite approximation through power three simplified to zero. The final column is the *returned* remainder power; this screen did not independently prove every such stronger remainder. All returned logarithmic degrees were zero.

Case	Source expression	Power
1	$\sin x/x^2$	5
2	$(e^x - 1 - x)/x^3$	4
3	$\log(1 + x)/x^4$	4
4	$(\sqrt{1 + x} - 1 - x/2)/x^3$	4
5	$1/(\sin x/x - 1)$	4
6	$(1 - \cos x)/(x - \sin x)$	5

Case	Source expression	Power
7	$\exp((\sin x - x)/x^3)$	4
8	$\log((1 - \cos x)/x^2)$	4
9	$(1 + \log(1 + x)/x)^{-1}$	4
10	$\sin(\log(1 + x)/x)$	4
11	$(1 + \sin x/x)^{\sqrt{2}}$	4
12	$((e^x - 1)/x)^{1/x}$	4
13	$(1 + x)^{1/x}$	4
14	$\exp(\sin x/x)$	4
15	$\sqrt{1 + x} - \sqrt{1 - x}$	5
16	$\log(1 + x) + \log(1 - x)$	4
17	$\exp(\log(1 + x))$	∞

B Ruled-out and excluded candidates

The guide explicitly defines the nonunit finite-endpoint observable power as $(x - x_0)^r$, while power one reconstructs x itself. The inspected Fourier construction follows that convention. Treating it as a lost translation would have been an incorrect new finding [4].

The attempted inexact-component request was refused in the native baseline. The tuple-validation patch therefore preserves and aligns exact-input checking; it is not presented as fixing a newly reproduced approximate-number escape. Likewise the ordinary observable multiplier example transported its remainder correctly. Negative candidates and successful controls are part of the audit record, not items to be silently discarded in order to increase the defect count.

Dense polynomial/Fourier support allocation, nonprincipal Mathics `ProductLog`, previously described validation-runner issues, and the nearby `SourceShift` error were already indexed. Their existing recommendations were not repeated. This boundary is recorded both here and in the novelty ledger so that the report remains an incremental contribution rather than another copy of the established backlog.

C Acceptance case map

ID	Desired post-patch contract
CV01	Refuse target dependence hidden by cancellation in the large-source split.
CV02	Refuse the corresponding <code>Abs[y]</code> spelling.
CV03	Refuse target dependence hidden by cancellation in the finite-source split.
CV04	Diagnose plain target dependence before an unrelated coefficient-realness refusal.
CV05	Refuse a target occurring only in the core.
CV06	Refuse a target occurring only in the perturbation.
CV07	Refuse identical source and target variables.
CV08	Preserve the unshifted depth-two inverse and its fifth-power error scale.

ID	Desired post-patch contract
CV09	Preserve a regular finite-endpoint inverse marker expansion.
CV10	Preserve a target-dependent supplied inverse for a fixed target-free core.
CV11	Preserve a fixed real symbolic parameter under its assumptions.
CV12	Preserve refusal of inexact input.

Only CV01, CV03, CV08, CV10, and the depth-one CV12 request were observed inline in patched native executions. The full twelve-case file and the four-case portable file remain supplied acceptance artifacts, not recorded passing suites.

References

- [1] Vladimir Reshetnikov, *Asymptotic repository*, pinned revision `efa1aee`, 10 September 2026 (full identifier on the title page). [\[commit-pinned source\]](#).
- [2] *CorePerturbation.wl*, same revision. Constructor admission at lines 126–127; model, marker recurrence, and remainder contract in the same file. [\[commit-pinned source\]](#).
- [3] *Kernel source map*, same revision. [\[commit-pinned source\]](#).
- [4] *AsymptoticAnalysis User Guide*, same revision. Entry points, Mathics instructions, power convention, and current coverage boundaries. [\[commit-pinned source\]](#).
- [5] *External code-review index*, same revision. [\[commit-pinned source\]](#).
- [6] *Code review implementation status*, same revision. [\[commit-pinned source\]](#).
- [7] *Wave 3 intake*, same revision. [\[commit-pinned source\]](#).
- [8] *Wave 4 intake*, same revision. [\[commit-pinned source\]](#).
- [9] *Wave 5 review index*, same revision. [\[commit-pinned source\]](#).
- [10] *Wave 6 review index*, same revision. [\[commit-pinned source\]](#).
- [11] *Code review 46: Target-dependent source shifts*, original report at its own baseline, retained in the pinned tree. README and article consulted. [\[commit-pinned source\]](#).
- [12] *Code review 43: Focused source-review deltas*, original report at its own baseline, retained in the pinned tree. F03 summarized in its README. [\[commit-pinned source\]](#).
- [13] Wolfram Research, *FreeQ*, Wolfram Language Documentation, accessed 10 September 2026. <https://reference.wolfram.com/language/ref/FreeQ.html>.
- [14] Wolfram Research, *Re*, Wolfram Language Documentation, accessed 10 September 2026. <https://reference.wolfram.com/language/ref/Re.html>.